

Programação Orientada a Objetos em Python

Encapsulamento



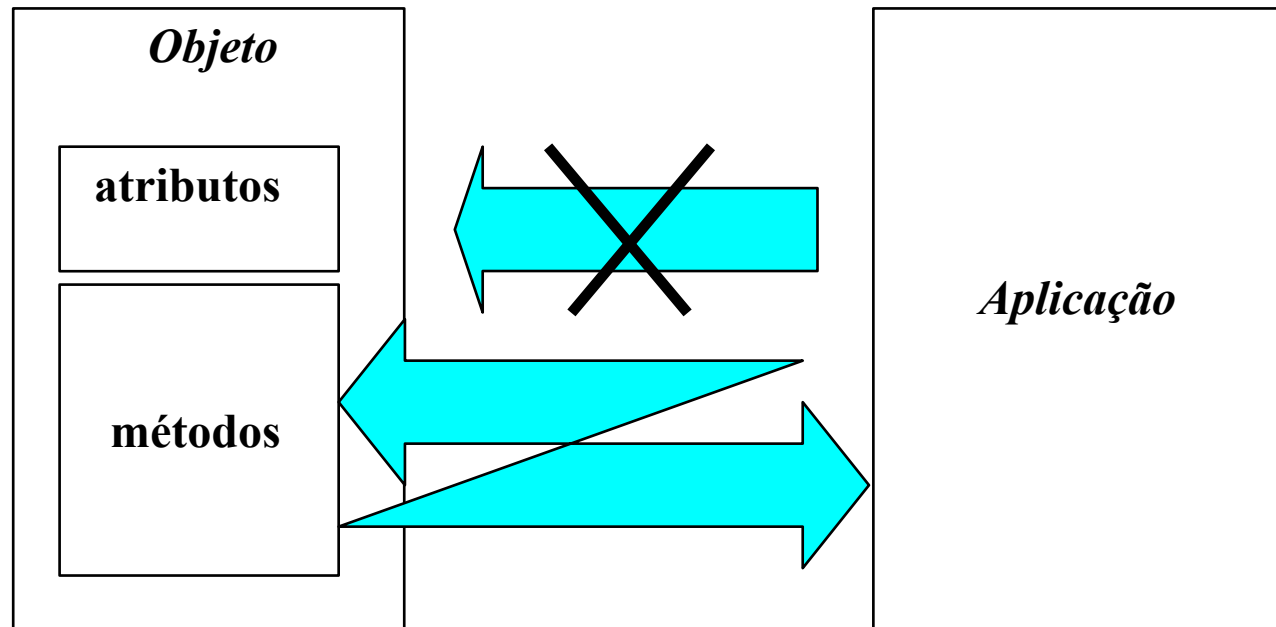
Prof. Maylon Henrique Oliveira
maylon.oliveira2@fatec.sp.gov.br

Cetec
Unidade do Ensino
Médio e Técnico

CPS
Centro
Paulista Souza


GOVERNO DO ESTADO
DE SÃO PAULO

Encapsulamento



Encapsulamento

Programação
Orientada a
Objetos

- ▶ Encapsulamento ou “*data hiding*” é um conceito bastante importante em orientação a objetos.
- ▶ É utilizado para restringir o acesso às declarações de uma classe e à própria classe.



Encapsulamento

Modificadores de Visibilidade

Programação
Orientada a
Objetos

- ▶ **public:** A classe, método ou atributo, são sempre acessíveis em todos os métodos de todas as classes.
- ▶ **private:** A classe, método ou atributo, são acessíveis somente por métodos da própria classe. É o nível mais rígido de encapsulamento.
- ▶ **protected:** A classe, método ou atributo, são acessíveis nos métodos da própria classe e suas subclasses (herança).
- ▶ No Python, diferente das linguagens completamente voltadas ao paradigma da orientação à objetos (Java, C#, etc.), estes atributos existem, mas não da forma “convencional”.



Encapsulamento

Exemplo

```
class Pessoa:
    def __init__(self, nome, idade, ativo):
        self.nome = nome
        self.idade = idade
        self.__ativo = ativo

    def desativar(self):
        self.__ativo = False
        print("A pessoa foi desativada")
```

Atributo privado (private)

```
if __name__ == "__main__":
    pessoa = Pessoa("Ana", 20, True)
    pessoa.desativar()
    pessoa.ativo = True
    print(pessoa.ativo)
```

Mesmo sendo privado, o Python permite o acesso fora da classe, pois se trata apenas de uma convenção.



Encapsulamento

Getters e Setters

- ▶ No Python, assim como em várias outras linguagens orientadas a objetos, há uma convenção de nomenclatura para os métodos que têm como finalidade acessar ou alterar as propriedades de um objeto.
- ▶ Segundo essa convenção, os nomes dos métodos que permitem a consulta das propriedades de um objeto devem possuir o prefixo ***get***. Analogamente, os nomes dos métodos que permitem a alteração das propriedades de um objeto devem possuir o prefixo ***set***.
- ▶ Na maioria dos casos, é muito conveniente seguir essa convenção, pois os desenvolvedores Python já estão acostumados com essas regras de nomenclatura.



Encapsulamento

Getters e Setters

- ▶ **set_Atributo(self, atributo)**: método utilizado para alterar o conteúdo de um atributo de uma classe, encapsulado com *private*;
- ▶ **get_Atributo(self)**: método utilizado para acessar o conteúdo de um atributo de uma classe, encapsulado com *private*;



Encapsulamento

Getters e Setters: Exemplo classe Pessoa

```
class Pessoa:
    def __init__(self, nome, idade, ativo):
        self.__nome = nome
        self.idade = idade
        self.__ativo = ativo

    def desativar(self):
        self.__ativo = False
        print("A pessoa foi desativada")

    def get_nome(self):
        return self.__nome

    def set_nome(self, nome):
        self.__nome = nome
```



Encapsulamento

Getters e Setters: Exemplo classe executora

```
if __name__ == "__main__":  
    pessoa = Pessoa("Ana", 20, True)  
    pessoa.desativar()  
    pessoa.ativo = True  
    print(pessoa.ativo)  
  
# Utilizando get e set  
    pessoa.set_nome("Maria")  
    print(pessoa.get_nome())
```



Encapsulamento

Getters e Setters: usando *Properties*

```
class Pessoa:
    def __init__(self, nome, idade, ativo):
        self.__nome = nome
        self.idade = idade
        self.__ativo = ativo

    def desativar(self):
        self.__ativo = False
        print("A pessoa foi desativada")

    @property # get
    def nome(self): return
        self.__nome

    @nome.setter #set
    def nome(self, nome):
        self.__nome = nome
```



Encapsulamento

Programação
Orientada a
Objetos

Getters e Setters: usando *Properties* classe executora

```
if __name__ == "__main__":  
    pessoa = Pessoa("Ana", 20, True)  
    pessoa.desativar()  
    pessoa.ativo = True  
    print(pessoa.ativo)  
  
# Utilizando properties  
pessoa.nome = "José"  
print(pessoa.nome)
```

